

MAPS — Material Allocation & Planning System

MAPS — система автоматического распределения строительных материалов на работы.

Система определяет, каким работам хватает материалов, а каким — нет, используя пятислойный алгоритм приоритизации. Результат — Excel-отчёт с полным разбором покрытия и дефицита.

Версия: 3.0.0 | Дата: 2026-05-23 | Python 3.11+ | PostgreSQL 15+

Содержание

1. [Зачем нужна система](#1-зачем-нужна-система)
 2. [Как работает алгоритм](#2-как-работает-алгоритм)
 3. [Архитектура проекта](#3-архитектура-проекта)
 4. [Технологический стек](#4-технологический-стек)
 5. [Установка и запуск](#5-установка-и-запуск)
 6. [Импорт данных из Excel](#6-импорт-данных-из-excel)
 7. [Веб-интерфейс и API](#7-веб-интерфейс-и-api)
 8. [AI-функции](#8-ai-функции)
 9. [CLI-команды](#9-cli-команды)
 10. [Экспорт результатов](#10-экспорт-результатов)
 11. [База данных](#11-база-данных)
 12. [Тесты](#12-тесты)
 13. [Деплой на сервер](#13-деплой-на-сервер)
 14. [Переменные окружения](#14-переменные-окружения)
 15. [Рoadmap](#15-roadmap)
 16. [Частые вопросы](#16-частые-вопросы)
 17. [Диагностика и решение проблем](#17-диагностика-и-решение-проблем)
-

1. Зачем нужна система

В строительно-монтажных работах одновременно выполняются десятки работ, каждая требует свой набор материалов. Вручную понять — хватит ли арматуры на все работы, или нужно срочно закупать — практически невозможно при больших объёмах.

До MAPS:

- Планировщик вручную сверяет Excel-таблицы с остатками и потребностями
- Ошибки: один и тот же материал может быть «распределён» дважды
- Нет учёта приоритетов: аварийные работы ждут в очереди наравне с плановыми
- Нет видимости дефицита: непонятно, чего не хватает и насколько

После MAPS:

- Алгоритм за секунды распределяет материалы по 5 слоям с учётом приоритетов
- Автоматически определяет дефицит и формирует список на закупку
- AI объясняет причины дефицита и прогнозирует будущие нехватки
- Excel-отчёт с полным разбором отправляется одной кнопкой

2. Как работает алгоритм

Алгоритм распределения работает по принципу «от факта к плану»: сначала учитывается то, что уже есть на объекте, затем остатки склада, и в последнюю очередь — закупка.

Порядок обработки работ

Работы обрабатываются в строгом порядке приоритетов:

1. Аварийные работы (`is_emergency=True`) — всегда первыми
2. По дате начала (`data_nachala`) — чем раньше старт, тем выше приоритет
3. По номеру приоритета (`prioritet: 1 > 2 > 3`)

Пять слоёв распределения

Для каждой пары «работа + материал» алгоритм последовательно проходит пять слоёв:

Слой 1: Фактические списания (<code>WriteOff</code>) Материал уже физически на объекте, документ SAP проведён. Источник: таблица <code>write_offs</code> Действие: уменьшает остаток потребности
Слой 2: Выдано не списано (<code>IssuedNotWrittenOff</code>) Материал выдан со склада, но документ ещё не проведён. Источник: таблица <code>issued_not_written_offs</code> Действие: уменьшает остаток потребности, склад НЕ трогает
Слой 3: Склад (<code>StockBatch</code>) Остатки на складах. Приоритет выдачи: 3а. Свой завод (совпадает <code>work.zavod = warehouse.zavod</code>) 3б. Свой филиал (<code>work.filial = warehouse.filial</code>) 3с. Прочие склады (другие филиалы) Партии внутри группы: FIFO (старые партии первыми) Действие: уменьшает <code>batch.dostupno</code>
Слой 4: Поставки (<code>SupplyLine</code>) Материал в пути (подтверждённые поставки). Только поставки для того же филиала. Действие: уменьшает <code>supply_line.dostupno</code>
Слой 5: Дефицит (<code>DeficitRecord</code>) Если после слоёв 1–4 потребность не покрыта полностью — остаток записывается как дефицит (нужно закупать).

Важные детали алгоритма

FIFO (First In First Out): Складские партии списываются от старых к новым по `data_postupleniya`. Это важно для корректного расчёта средневзвешенной стоимости.

Взвешенная стоимость: Если со склада берётся из нескольких партий по разной цене, в результате записывается средневзвешенная стоимость.

Анализ межфилиального покрытия: Алгоритм дополнительно пометает, сколько можно было бы взять с других филиалов (`is_possible=True`). Это не меняет реальных остатков, но показывает потенциал перемещения.

Сброс перед каждым запуском: `reset_dostupno()` восстанавливает `dostupno = kolichestvo` для всех партий и поставок перед новым запуском. Это гарантирует что повторный запуск даёт тот же результат, что и первый.

3. Архитектура проекта

```
PythonProject/
├── app/                                     # Основной пакет приложения
│   ├── allocation/
│   │   └── engine.py                     # Ядро алгоритма распределения (5 слоёв)
│   ├── ai/                               # AI-модули (Claude API)
│   │   ├── deficit_analyzer.py          # Объяснение причин дефицита
│   │   ├── deficit_forecaster.py        # (не используется)
│   │   └── session_rag.py               # (не используется)
│   ├── api/                              # FastAPI веб-приложение
│   │   ├── main.py                     # Точка входа FastAPI, регистрация роутеров
│   │   └── routes/
│   │       ├── status.py                # GET /api/status
│   │       ├── allocation.py            # POST /api/allocate
│   │       ├── import_.py               # POST /api/import/* (6 типов файлов)
│   │       ├── export.py                # GET /api/export/current
│   │       └── ai.py                    # POST /api/ai/explain
│   └── templates/
│       └── index.html                   # Bootstrap 5 дашборд (SPA на fetch API)
├── core/
│   ├── config.py                        # Настройки через pydantic-settings (.env)
│   └── logging_config.py                 # Конфигурация логирования
├── db/
│   ├── models.py                       # SQLAlchemy ORM-модели (все таблицы)
│   └── database.py                      # Движок, сессии, get_session()
├── models/
│   └── schemas.py                       # Pydantic-схемы валидации (импорт/экспорт)
├── repositories/
│   ├── work_repository.py               # Запросы: приоритизация, сброс
│   └── material_repository.py           # Запросы: остатки по складам
└── services/
```

└─ import_service.py	# Импорт Excel → БД (6 типов)
└─ export_service.py	# Экспорт результатов → Excel
└─ deploy/	# Скрипты деплоя на Oracle Cloud
└─└─ setup.sh	# Установка всего на сервере одной командой
└─└─ maps.service	# systemd-юнит (автозапуск)
└─└─ tunnel.sh	# Cloudflare Quick Tunnel
└─└─ README.md	# Инструкция деплоя
└─ migrations/	# Alembic миграции базы данных
└─└─ versions/	# SQL-скрипты изменений схемы
└─ tests/	# Автоматические тесты (38 тестов)
└─└─ conftest.py	# Фикстуры: БД, тестовые данные
└─└─ test_engine.py	# 17 тестов алгоритма распределения
└─└─ test_repositories.py	# 13 тестов репозитория
└─└─ test_allocation.py	# 8 интеграционных тестов
└─ data/	# Excel-файлы для импорта и экспорта
└─└─ exports/	# Сгенерированные отчёты
└─ main.py	# CLI точка входа (typer)
└─ pyproject.toml	# Метаданные и зависимости проекта
└─ requirements.txt	# pip зависимости
└─ alembic.ini	# Конфигурация Alembic
└─ .env	# Переменные окружения (не в git!)

4. Технологический стек

Компонент	Технология	Зачем
База данных	PostgreSQL 15+	Надёжность, параллельность, bulk UPDATE
ORM	SQLAlchemy 2.0	Работа с БД через Python-классы
Миграции	Alembic	Изменения схемы без потери данных
Валидация	Pydantic 2	Проверка данных при импорте и в API
Web-фреймворк	FastAPI 0.110+	REST API, автодокументация, async
Web-сервер	uvicorn	ASGI-сервер для FastAPI
Шаблоны	Jinja2	HTML дашборд
Файлы	multipart + openpyxl	Загрузка и генерация Excel
CLI	typer + rich	Красивый терминальный интерфейс
AI	Anthropic Claude API	Анализ причин дефицита

Компонент	Технология	Зачем
Данные	pandas	Чтение Excel, обработка таблиц
Тесты	pytest	38 автоматических тестов
Линтер	ruff	Проверка стиля кода

5. Установка и запуск

Требования

- Python 3.11+
- PostgreSQL 15+
- pip

Установка

```
# Клонировать репозиторий
git clone https://github.com/DaniarSher81212/maps.git && cd maps
# Создать виртуальное окружение
python3.11 -m venv .venv
source .venv/bin/activate      # Linux/Mac
# .venv\Scripts\activate      # Windows
# Установить зависимости
pip install -e ".[dev]"        # dev = тесты + линтер
# или только основные:
pip install -r requirements.txt
# Создать .env (скопировать шаблон и заполнить)
cp .env.example .env
nano .env
```

Настройка базы данных

```
# Создать БД и пользователя в PostgreSQL
sudo -u postgres psql -c "CREATE USER maps_user WITH PASSWORD 'your_password';"
sudo -u postgres psql -c "CREATE DATABASE maps_db OWNER maps_user;"
# Применить миграции (создать таблицы)
alembic upgrade head
```

Запуск

```
# Веб-сервер (рекомендуется)
maps serve
# С дополнительными параметрами
maps serve --host 0.0.0.0 --port 8080 --reload
# Напрямую через uvicorn (для разработки)
uvicorn app.api.main:app --reload
```

После запуска откройте <http://localhost:8000>

6. Импорт данных из Excel

Все данные загружаются через веб-интерфейс или API. Поддерживается шесть типов файлов.

Типы файлов и их назначение

Файл	Эндпоинт	Что содержит
Потребности	POST /api/import/requirements	Работы и их потребности в материалах
Аварийные работы	POST /api/import/emergency	Аварийные работы (высший приоритет)
Складские остатки	POST /api/import/stock	Партии материалов на складах
Поставки	POST /api/import/supplies	Материалы в пути (ожидаемые поставки)
Списания	POST /api/import/writeoffs	Фактически списанные на работы (Слой 1)
Выдано не списано	POST /api/import/issued	Выдано, но документ не проведён (Слой 2)

Рекомендуемый порядок загрузки

1. Потребности (requirements) – основа: работы и что им нужно
2. Складские остатки (stock) – что есть на складах
3. Поставки (supplies) – что ожидается
4. Списания (writeoffs) – что уже на объекте (документально)
5. Выдано не списано (issued) – что выдано, но не оформлено
6. Аварийные работы (emergency) – аварийные работы (если есть)

Загрузка через UI

На дашборде (правая панель) раскройте нужный аккордеон → выберите файл → импорт начинается автоматически. Отображается статистика: сколько строк загружено, обновлено, с ошибками.

Загрузка через curl

```
# Загрузить потребности
curl -X POST http://localhost:8000/api/import/requirements \
  -F "file=@data/requirements.xlsx"
# Загрузить остатки склада
curl -X POST http://localhost:8000/api/import/stock \
  -F "file=@data/stock.xlsx"
```

Формат Excel-файлов

Файлы читаются через pandas. Названия колонок нормализуются автоматически. Минимальные колонки для потребностей:

kod_raboty	sys_nomer_materiala	potrebnost
WO-001	10012345	100
WO-002	10012346	50.5

7. Веб-интерфейс и API

Дашборд

Откройте <http://localhost:8000> — Bootstrap 5 интерфейс:

Статус-карточки (вверху):

- Количество активных работ
- Количество материалов в справочнике
- Количество партий с ненулевым остатком
- Статус и время последнего запуска

Левая панель — Запуск:

- Кнопка «Запустить распределение» — запускает алгоритм в фоне
- Карточка текущего результата (статус, потребностей, дефицит)
- Кнопка «Скачать Excel» для текущей сессии

Правая панель — Импорт файлов:

- Аккордеон с 6 формами загрузки
- Выбор файла → автоматический импорт → статистика результата

AI-секция:

- Выбор материала из дефицита → AI объясняет причину нехватки

REST API

Метод	URL	Описание
GET	/	HTML дашборд
GET	/api/status	Статус системы: счётчики, текущая сессия
POST	/api/allocate	Запустить распределение (фоновая задача)
POST	/api/import/requirements	Импорт потребностей (+ полный сброс БД)
POST	/api/import/emergency	Импорт аварийных работ
POST	/api/import/stock	Импорт остатков склада
POST	/api/import/supplies	Импорт поставок
POST	/api/import/writeoffs	Импорт списаний

Метод	URL	Описание
POST	/api/import/issued	Импорт выдано не списано
GET	/api/export/{session_id}	Скачать Excel-отчёт текущей сессии
GET	/api/ai/deficits/{session_id}	Список материалов с дефицитом
POST	/api/ai/explain	AI-объяснение причины дефицита
GET	/docs	Swagger UI
GET	/redoc	ReDoc

Примеры curl

```
# Статус системы
curl http://localhost:8000/api/status
# Запустить распределение
curl -X POST http://localhost:8000/api/allocate
# Скачать отчёт текущей сессии
curl -O -J http://localhost:8000/api/export/20260523_184619_66b95
# AI: объяснить дефицит по материалу
curl -X POST http://localhost:8000/api/ai/explain \
  -H "Content-Type: application/json" \
  -d '{"session_id": "20260523_184619_66b95", "material_id": 5}'
```

Временные ссылки для скачивания с другого устройства

Если MAPS запущен на домашнем компьютере (192.168.1.100):

```
http://192.168.1.100:8000/api/export/{session_id}
```

Или через Cloudflare Tunnel (публичная ссылка):

```
bash deploy/tunnel.sh
# Получите: https://xxxx.trycloudflare.com
# Отчёт:    https://xxxx.trycloudflare.com/api/export/{session_id}
```

8. AI-функции

AI-функция работает через Claude API (Anthropic). Требуется ANTHROPIC_API_KEY в .env. Без ключа система работает полностью, кроме AI-анализа.

Анализ причины дефицита

Объясняет на русском языке, почему конкретный материал оказался в дефиците. Разбирает каждый слой: что покрыло, чего не хватило, какие остатки были.

Через UI: Дашборд → «AI-анализ» → выбрать материал → «Объяснить»

Через API:

```
# 1. Список материалов с дефицитом в текущей сессии
```



```
curl http://localhost:8000/api/ai/deficits/20260523_184619_66b95
# 2. Объяснение
curl -X POST http://localhost:8000/api/ai/explain \
  -H "Content-Type: application/json" \
  -d '{"session_id": "20260523_184619_66b95", "material_id": 5}'
```

Пример ответа:

Главная причина дефицита — полное отсутствие материала по всем источникам покрытия. Ни на складах завода KZ04 (Актобе), ни в рамках филиала нет ни одного мешка цемента М400. Две работы с суммарной потребностью 100 мешков стартуют уже 24.05.2026. Рекомендация: экстренная закупка минимум 100 мешков сегодня.

9. CLI-команды

После установки (`pip install -e .`) доступна команда `maps`:

```
# Запустить веб-сервер
maps serve
# С параметрами
maps serve --host 0.0.0.0 --port 8080 --reload
# Импорт данных (порядок важен; import-requirements делает полный сброс БД)
maps import-requirements data/requirements.xlsx
maps import-emergency data/emergency.xlsx
maps import-stock data/stock.xlsx
maps import-supplies data/supplies.xlsx
maps import-writeoffs data/writeoffs.xlsx
maps import-issued data/issued.xlsx
# Запустить распределение
maps allocate
# Просмотр текущих результатов
maps show
# Экспорт текущей сессии в Excel
maps export
# Состояние БД (счётчики таблиц)
maps status
# Полный сброс всех данных
maps reset-data
# Справка
maps --help
```

Параметр <code>serve</code>	По умолчанию	Описание
<code>--host</code>	0.0.0.0	IP-адрес для прослушивания
<code>--port</code>	8000	Порт
<code>--reload</code>	False	Авто-перезапуск при изменении кода

10. Экспорт результатов

После запуска распределения результаты скачиваются в Excel.

Структура Excel-отчёта

Лист	Содержимое
Распределение	Детальные результаты: работа, материал, источник, количество, стоимость
Дефицит	Дефицитные позиции: что, сколько, для какой работы, дата нужна к
Сводка	Агрегированная статистика по филиалам и материалам

Скачать отчёт

```
# Через CLI
maps export
# Через curl (ID текущей сессии – из maps status или maps show)
curl -O -J http://localhost:8000/api/export/20260523_184619_66b95
# Через браузер
http://localhost:8000/api/export/20260523_184619_66b95
```

11. База данных

Таблицы

Таблица	Описание
works	Строительные работы (код, филиал, завод, приоритет, даты)
materials	Справочник материалов (sys_pomer, название, ед.изм.)
requirements	Потребности работ в материалах (связь work ↔ material)
warehouses	Склады (код, филиал, завод, тип)
stock_batches	Партии материалов на складах (FIFO по data_postupleniya)
supplies	Заказы на поставку (договор, поставщик, дата)
supply_lines	Строки поставок (материал, количество, dostupno)
write_offs	Фактические списания на работы (Слой 1)
issued_not_written_offs	Выдано не списано (Слой 2)
allocation_sessions	Сессии запуска алгоритма (id, статус, время)
allocation_results	Результаты распределения по слоям
deficit_records	Записи о дефиците (Слой 5)

Ключевые поля

`stock_batches.dostupno` — доступный остаток. Уменьшается при распределении. Восстанавливается через `reset_dostupno()` перед каждым новым запуском.

`allocation_results.istochnik` — источник покрытия:

- "spisanie" — Слой 1: фактические списания
- "vydano" — Слой 2: выдано не списано
- "sklad" — Слой 3: со склада
- "postavka" — Слой 4: из поставки

`allocation_results.is_possible` — если True, это аналитическая запись (возможное межфилиальное покрытие), реальные остатки НЕ изменялись.

Миграции

```
# Применить все миграции
alembic upgrade head
# Откатить последнюю миграцию
alembic downgrade -1
# Создать новую миграцию (после изменения models.py)
alembic revision --autogenerate -m "описание изменения"
# История миграций
alembic history
```

12. Тесты

38 автоматических тестов покрывают алгоритм, репозитории и интеграцию.

Запуск

```
# Все тесты
pytest
# С отчётом покрытия
pytest --cov=app --cov-report=term-missing
# Конкретный файл
pytest tests/test_engine.py -v
# Конкретный тест
pytest tests/test_engine.py::test_layer1_full_writeoff -v
```

Структура тестов

`tests/conftest.py` — фикстуры:

- `engine` — тестовая БД (PostgreSQL, та же схема что и prod)
- `connection` — соединение с транзакцией (откатывается после теста)
- `session` — SQLAlchemy-сессия, привязанная к транзакции
- `base_data` — базовый набор тестовых данных

Каждый тест изолирован: транзакция откатывается после завершения, данные не накапливаются.

tests/test_engine.py — 17 тестов алгоритма:

- Слой 1: полное списание, частичное, средневзвешенная стоимость
- Слой 2: покрытие выданным, берёт только остаток
- Слой 3: приоритет своего завода, FIFO, межфилиальный fallback
- Слой 4: покрытие поставкой, только свой филиал
- Слой 5: полный дефицит, частичный дефицит
- Аварийный приоритет, повторный запуск (тест reset_dostupno)

tests/test_repositories.py — 13 тестов репозитория:

- Порядок работ (аварийные первыми, по дате, по приоритету)
- Сброс raspredeleno и дефицитов
- Группировка склада (zavod / filial / prochee)

tests/test_allocation.py — 8 интеграционных тестов:

- Полный цикл: импорт → распределение → проверка результатов

13. Деплой на сервер

Текущий production-сервер

Параметр	Значение
Провайдер	Oracle Cloud (Always Free tier)
IP-адрес	132.145.232.46
ОС	Ubuntu 22.04 LTS
Пользователь	ubuntu
Путь проекта	~/maps
Автозапуск	systemd-сервис maps.service
Публичный доступ	Cloudflare Quick Tunnel

Шаг 1 — Подключиться к серверу

Oracle Cloud требует SSH-ключ (пароли отключены). Два способа:

Вариант А — Oracle Cloud Shell (рекомендуется, не нужен ключ)

Откройте cloud.oracle.com → нажмите иконку >_ (Cloud Shell) в правом верхнем углу браузера. Откроется встроенный терминал с уже настроенным доступом к вашим инстансам.

```
# В Oracle Cloud Shell:  
ssh ubuntu@132.145.232.46
```

Вариант В — С локальной машины (нужен SSH-ключ)

```
# Создать ключ (один раз)
ssh-keygen -t ed25519 -f ~/.ssh/maps_key -N ""
# Показать публичный ключ – скопируйте его
cat ~/.ssh/maps_key.pub
```

Добавьте публичный ключ в Oracle Cloud Console:

Compute → Instances → ваш инстанс → Add SSH Keys → вставьте ключ

```
# Подключиться
ssh -i ~/.ssh/maps_key ubuntu@132.145.232.46
```

Шаг 2 — Клонировать проект на сервер

```
# На сервере (после SSH):
git clone https://github.com/DaniarSher81212/maps.git
cd maps
```

Шаг 3 — Запустить установку

```
sudo bash deploy/setup.sh
```

Что делает скрипт (~5 минут):

Действие	Результат
apt-get install python3.11 postgresql cloudflared	Системные зависимости
CREATE USER maps_user + CREATE DATABASE maps_db	PostgreSQL готов
python3.11 -m venv .venv && pip install -e .	Виртуальное окружение
Создаёт .env с параметрами production	Конфигурация приложения
alembic upgrade head	Таблицы созданы в БД
systemctl enable maps	Сервис зарегистрирован, автозапуск включён

По окончании выводится итоговое сообщение с командами управления.

Шаг 4 — Добавить ANTHROPIC_API_KEY

setup.sh создаёт .env без ключа Claude API (ключ нельзя хранить в скрипте). Добавьте вручную:

```
echo "ANTHROPIC_API_KEY=sk-ant-api03-ВАШ_КЛЮЧ" >> .env
```

Ключ можно получить или скопировать на
console.anthropic.com.

*Без ключа система работает — импорт, распределение, экспорт. Только три
AI-эндпоинта вернут 503.*

Шаг 5 — Запустить MAPS и проверить

```
sudo systemctl start maps
# Проверить статус — должно быть "Active: active (running)"
sudo systemctl status maps
# Проверить что API отвечает
curl http://localhost:8000/api/status
```

Шаг 6 — Открыть Cloudflare Quick Tunnel

```
# Фоновый режим: туннель работает даже при закрытом терминале*
bash deploy/tunnel.sh --bg
# Получить публичный URL
cat /tmp/maps_tunnel.url
# → https://xxxx.trycloudflare.com
```

Откройте ссылку в браузере — должен загрузиться дашборд MAPS.

**При закрытии Cloud Shell сессии туннель всё равно остановится. Чтобы туннель работал постоянно — добавьте его в systemd (см. ниже).*

Управление сервисом

sudo systemctl start maps	# Запустить
sudo systemctl stop maps	# Остановить
sudo systemctl restart maps	# Перезапустить (после обновления кода)
sudo systemctl status maps	# Статус + последние строки лога
sudo systemctl enable maps	# Включить автозапуск (уже включён после setup.sh)
sudo systemctl disable maps	# Отключить автозапуск

Просмотр логов

```
# Логи в реальном времени (Ctrl+C для выхода)
sudo journalctl -u maps -f
# Последние 50 строк
sudo journalctl -u maps -n 50 --no-pager
# Логи за последний час
sudo journalctl -u maps --since "1 hour ago" --no-pager
# Логи с момента последнего старта
sudo journalctl -u maps -b --no-pager
```

Обновление кода (после git push)

```
cd ~/maps
git pull origin main
sudo systemctl restart maps
sudo systemctl status maps # Убедиться что запустился
```

Cloudflare Quick Tunnel

Cloudflare Tunnel создаёт публичный HTTPS-адрес без необходимости открывать порты в Oracle Cloud или иметь домен. Всё работает через исходящие соединения (outbound), поэтому фаервол настраивать не нужно.

```
# Запуск в foreground (туннель работает пока открыт терминал):
bash deploy/tunnel.sh
# Запуск в фоне (URL сохраняется в файл):
bash deploy/tunnel.sh --bg
cat /tmp/maps_tunnel.url
# Остановить фоновый туннель:
kill $(cat /tmp/cloudflared.pid)
```

Особенности Quick Tunnel:

- URL меняется при каждом перезапуске (это бесплатная версия)
- Если нужен постоянный URL — нужна регистрация в Cloudflare и настройка Named Tunnel
- Туннель работает только пока запущен процесс cloudflared

Постоянный туннель через systemd (опционально)

Чтобы туннель запускался автоматически вместе с сервером:

```
# Создать systemd-сервис для туннеля
sudo tee /etc/systemd/system/maps-tunnel.service > /dev/null <<EOF
[Unit]
Description=MAPS Cloudflare Tunnel
After=maps.service
Requires=maps.service
[Service]
Type=simple
User=ubuntu
ExecStart=/usr/local/bin/cloudflared tunnel --url http://localhost:8000 --no-autoupdate
Restart=on-failure
RestartSec=10s
StandardOutput=journal
StandardError=journal
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable maps-tunnel
sudo systemctl start maps-tunnel
# Посмотреть URL в логах
sudo journalctl -u maps-tunnel -n 20 | grep trycloudflare
```

Известные особенности Oracle Cloud

Проблема	Причина	Решение
Permission denied (publickey) при SSH	Oracle Cloud отключает вход по паролю	Используйте Oracle Cloud Shell или добавьте SSH-ключ
ModuleNotFoundError: No module named 'anthropic' при запуске	Пакет не попал в установку	<code>.venv/bin/pip install anthropic</code>

Проблема	Причина	Решение
Сервис запущен, но curl / возвращает 404	Воркеры uvicorn падают при старте	Убрать --workers 2 из maps.service (уже сделано)
Порт 8000 занят после systemctl stop	Воркеры не завершились	sudo fuser -k 8000/tcp

14. Переменные окружения

Файл .env в корне проекта. Никогда не коммитьте .env в git.

```
cp .env.example .env
nano .env
```

Переменная	Обязательна	По умолчанию	Описание
DB_HOST	Нет	localhost	Хост PostgreSQL
DB_PORT	Нет	5432	Порт PostgreSQL
DB_NAME	Нет	maps_db	Имя базы данных
DB_USER	Нет	maps_user	Пользователь БД
DB_PASSWORD	Да	—	Пароль пользователя БД
DATABASE_URL	Нет	—	Полный URL (переопределяет DB_* поля)
APP_ENV	Нет	development	development / production
LOG_LEVEL	Нет	INFO	DEBUG / INFO / WARNING / ERROR
SESSION_PREFIX	Нет	session	Префикс ID сессий
DATA_DIR	Нет	data	Папка для Excel-файлов
EXPORT_DIR	Нет	data/exports	Папка для отчётов
ANTHROPIC_API_KEY	Для AI	—	Ключ Claude API (console.anthropic.com)

Важно: Без ANTHROPIC_API_KEY система работает полностью — импорт, распределение, экспорт. Только три AI-эндпоинта возвращают 503 с понятным сообщением.

15. Рoadmap

📌 Этап 1 — Ядро системы (выполнено)

- [x] Пятислойный алгоритм распределения с FIFO и взвешенной стоимостью
- [x] Импорт данных из 6 типов Excel-файлов
- [x] Экспорт результатов в Excel (3 листа: распределение, дефицит, сводка)
- [x] PostgreSQL + SQLAlchemy 2.0 + Alembic миграции
- [x] CLI-интерфейс (typer + rich)
- [x] 38 автоматических тестов с изолированными транзакциями

□ Этап 2 — Веб-интерфейс (выполнено)

- [x] FastAPI REST API (18 эндпоинтов)
- [x] Bootstrap 5 дашборд с JavaScript polling каждые 3 сек
- [x] Фоновый запуск распределения (BackgroundTasks)
- [x] Загрузка файлов через веб-форму с мгновенной статистикой
- [x] Временные ссылки для скачивания отчётов

□ Этап 3 — Деплой и AI (выполнено)

- [x] Деплой на Oracle Cloud (setup.sh + systemd-сервис)
- [x] Cloudflare Quick Tunnel без домена и регистрации
- [x] AI-анализ причин дефицита (Claude Sonnet)
- [x] AI-анализ причин дефицита (Claude Sonnet)

□ Этап 4 — Расширение (следующий)

- [] CI/CD через GitHub Actions (тесты при каждом push)
- [] Фильтрация распределения по конкретному филиалу/заводу
- [] Сценарное моделирование (именованные версии распределения)
- [] Расширенный аудит (кто запустил, какие файлы загрузил)
- [] Интеграция с SAP (чтение данных через SAP GUI Scripting)

□ Этап 5 — Оптимизация

- [] Критический путь (CPM — Critical Path Method)
- [] Граф конфликтов ресурсов (NetworkX)
- [] Оптимизация распределения (Google OR-Tools)
- [] Анализ рисков поставок (вероятность задержки)

16. Частые вопросы

Q: Почему PostgreSQL, а не SQLite?

PostgreSQL рассчитан на параллельную работу нескольких пользователей. SQLite блокирует базу при записи — при одновременном импорте и распределении были бы конфликты. Кроме того, MAPS использует bulk UPDATE и оконные функции которые быстрее работают в PostgreSQL.

Q: Почему алгоритм запускается как BackgroundTask?

HTTP-запрос должен вернуть ответ быстро (до таймаута браузера ~30 сек). Распределение на больших объёмах данных может занять несколько минут.

BackgroundTask возвращает {"status": "started"} немедленно, а дашборд опрашивает /api/status каждые 3 секунды и автоматически обновляет статус.

Q: Что будет если запустить распределение дважды подряд?

Второй запуск полностью заменит результаты первого. Перед каждым запуском удаляется предыдущая сессия и восстанавливаются остатки (dostupno = kolichestvo). В базе всегда хранится только одна (текущая) сессия.

Q: Как работает FIFO?

При распределении со склада партии сортируются по data_postupleniya (дата поступления) от старых к новым. Сначала уходят партии с более ранней датой. Если даты одинаковые — по id (порядку создания в БД).

Q: Что такое is_possible в результатах?

При анализе Слой 3 алгоритм проверяет все склады, включая другие филиалы. Если на другом филиале есть материал, создаётся запись с is_possible=True. Это аналитическая информация: «можно было бы взять отсюда при межфилиальном перемещении». Реальные остатки при этом не изменяются.

Q: Нужен ли Anthropic API для работы системы?

Нет. Без ANTHROPIC_API_KEY всё работает: импорт, распределение, экспорт. Только AI-эндпоинт /api/ai/explain вернёт 503 с сообщением «ANTHROPIC_API_KEY не задан в .env».

Q: Почему ANTHROPIC_API_KEY нельзя хранить в коде?

Ключ — это пароль. Если добавить его в код и запустить в git, он будет виден всем кто имеет доступ к репозиторию. Нужно хранить только в .env, который добавлен в .gitignore. Если ключ случайно засветился — сразу идите на console.anthropic.com и отзывайте его.

17. Диагностика и решение проблем

MAPS не запускается

```
# Смотрим детальный лог ошибки:
sudo journalctl -u maps -n 100 --no-pager
# Тест: сам Python может импортировать приложение?
cd ~/maps && .venv/bin/python -c "from app.api.main import app; print('OK!)"
# Тест: uvicorn запускается вручную?
sudo systemctl stop maps
sudo fuser -k 8000/tcp          # освободить порт
.venv/bin/uvicorn app.api.main:app --host 0.0.0.0 --port 8000
```

Ошибка: ModuleNotFoundError

```
ModuleNotFoundError: No module named 'anthropic'
ModuleNotFoundError: No module named 'fastapi'
```

Причина: `pip install -e .` установил не все пакеты (они были в requirements.txt, но не в pyproject.toml).

```
# Установить недостающие пакеты:
.venv/bin/pip install anthropic fastapi uvicorn[standard] jinja2 python-multipart
sudo systemctl restart maps
```

Ошибка: address already in use

ERROR: [Errno 98] error while attempting to bind on address ('0.0.0.0', 8000): address already in use

Причина: Предыдущий процесс uvicorn не завершился.

```
# Найти и убить процесс на порту 8000:
sudo fuser -k 8000/tcp
# Или найти PID вручную:
sudo ss -tlnp | grep 8000
kill <PID>
```

Сервис запущен, но дашборд отдаёт 404

Причина: Воркеры uvicorn падают при старте, master-процесс жив но маршруты не работают.

```
# Проверить количество задач (с --workers 2 должно быть 3+, иначе воркеры упали):
sudo systemctl status maps | grep Tasks
# Проверить что установлен режим без --workers:
grep ExecStart /etc/systemd/system/maps.service
# Если там всё ещё --workers 2 — убрать:
sudo sed -i '/--workers/d' /etc/systemd/system/maps.service
sudo systemctl daemon-reload
sudo systemctl restart maps
```

Cloudflare Tunnel не запускается

```
# Проверить что MAPS отвечает:
curl http://localhost:8000/api/status
# Запустить туннель вручную и смотреть вывод:
cloudflared tunnel --url http://localhost:8000 --no-autoupdate
# Если cloudflared не установлен:
ARCH=$(dpkg --print-architecture)
curl -L "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-${ARCH}.deb" -o /
sudo dpkg -i /tmp/cf.deb
```

Ошибка подключения к PostgreSQL

sqlalchemy.exc.OperationalError: could not connect to server

```
# Проверить что PostgreSQL запущен:
sudo systemctl status postgresql
# Проверить подключение вручную:
psql -U maps_user -d maps_db -h localhost -c "SELECT 1;"
# Проверить что пароль в .env совпадает с паролем в PostgreSQL:
cat .env | grep DB_PASSWORD
# Сменить пароль если нужно:
sudo -u postgres psql -c "ALTER USER maps_user PASSWORD 'новый_пароль';"
# И обновить в .env
nano .env
sudo systemctl restart maps
```

Ошибки при импорте Excel

```
KeyError: 'kod_raboty'          # колонка не найдена  
ValueError: could not convert  # неверный тип данных
```

Диагностика:

```
# Посмотреть что пришло в API:  
sudo journalctl -u maps -n 30 --no-pager | grep -E "(ERROR|error|import)"
```

Типичные причины:

- Другое название колонки (пробелы, другой регистр) — `import_service` нормализует, но только известные варианты
- Пустые строки в Excel — игнорируются автоматически
- Числа в колонке-строке — `Pydantic` сконвертирует, если возможно

Тесты падают

```
# Убедиться что тестовая БД доступна:  
psql -U maps_user -d maps_db -c "SELECT 1;"  
# Запустить один тест с подробным выводом:  
pytest tests/test_engine.py::test_layer1_full_writeoff -v -s  
# Посмотреть фикстуры (conftest.py) — там создаётся тестовое окружение:  
cat tests/conftest.py
```

Общий чеклист диагностики

```
# 1. Статус сервиса  
sudo systemctl status maps  
# 2. Последние логи  
sudo journalctl -u maps -n 30 --no-pager  
# 3. Приложение отвечает?  
curl http://localhost:8000/api/status  
# 4. PostgreSQL работает?  
sudo systemctl status postgresql  
# 5. Свободное место на диске (pandas + Excel могут занять много)  
df -h  
# 6. Свободная память  
free -h  
# 7. Версия Python в venv  
.venv/bin/python --version  
# 8. Установленные пакеты  
.venv/bin/pip list | grep -E "(anthropic|fastapi|uvicorn|sqlalchemy)"
```